DAY 05

Functions & Scope

Beginner Phase · 2 Hours · Session Guide

■ **Day**	05 of 30
■ **Phase**	Beginner (Days 2–6)
■ **Duration**	2 hours (120 minutes)
■ **Topic**	Functions & Scope
■ **Prerequisites**	Days 1–4 complete

■ **Learning Objectives**

- Define and call functions using the def keyword
- Work with parameters, arguments, and return values
- Use default arguments, *args, and **kwargs
- Understand Python scope and the LEGB rule
- Write clean, reusable, well-structured functions

■ SECTION 2 — SESSION OVERVIEW

Session Goal

Students will understand how to define reusable blocks of code as functions, pass data in and out, and reason about where variables live using Python's LEGB scope rules.

Session Outcome

By the end of this session, students can write multi-function Python scripts, confidently use *args and **kwargs, and explain why a variable is or isn't accessible in a given context.

Minute-by-Minute Timeline

Time	Activity	Type
0:00 – 0:10	Recap: loops and control flow Q&A	Review
0:10 – 0:25	Concept: def, parameters, return	Lecture
0:25 – 0:40	Live coding: building a calculator	Demo
0:40 – 0:55	Default args, *args, **kwargs	Lecture + Demo
0:55 – 1:10	Student exercises (functions)	Practice
1:10 – 1:25	Concept: scope and LEGB rule	Lecture
1:25 – 1:40	Live coding: scope gotchas	Demo
1:40 – 1:55	Mini-project: tip calculator	Practice
1:55 – 2:00	Wrap-up, checklist, preview Day 6	Wrap-up

■ Instructor Note

Functions feel abstract until students see them solving real duplication problems.
Open with: 'What if you needed the same 10 lines in 5 places?' — then show the fix.
Scope is the #1 confusion point. Use a whiteboard diagram of LEGB boxes.

■ SECTION 3 — PREREQUISITES & PREPARATION

Student Prerequisites

Topic	Where Covered	Required?
Variables & data types	Day 2	■ Yes
Control flow (if/elif)	Day 3	■ Yes
Loops (for/while)	Day 4	■ Yes
Virtual environment	Day 1	■ Yes
VS Code setup	Day 1	■ Yes

Instructor Pre-Session Checklist

■ Open VS Code with a fresh day05_functions.py file

■ Have a browser tab open at docs.python.org/3/tutorial/controlflow.html

■ Prepare whiteboard / slide showing LEGB scope diagram

■ Test that all code snippets in this guide run error-free

■ Review student exercise solutions in the practice notebook

■ SECTION 4 — STEP-BY-STEP INSTRUCTION GUIDE

Step 1 — Defining Functions with def (0:10 – 0:25)

What to Say:

"So far, when we wanted to repeat logic we used a loop. But what if the same logic is needed in different places, with different data? That's where functions come in. A function is a named, reusable block of code. You define it once and call it as many times as you need."

Live Code — Basic Function

```
# ■■ Defining a function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def greet(name):                    # 'name' is a parameter
    message = f"Hello, {name}!"     # local variable
    return message                  # send value back to caller

# ■■ Calling the function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
result = greet("Priya")             # "Priya" is an argument
print(result)                       # Hello, Priya!
print(greet('Arjun'))               # Hello, Arjun!
```

Key Vocabulary Table:

Term	Meaning	Example
def	Keyword that starts a function definition	def greet():
parameter	Variable placeholder in the definition	name in greet(name)
argument	Actual value passed when calling	"Priya" in greet("Priya")
return	Sends a value back to the caller	return message
call	Executing the function	greet('Arjun')

Common Errors — Step 1:

Error	Cause	Fix
IndentationError	Body not indented	Indent 4 spaces
NameError: 'greet' not defined	Called before def block	Move def above call
Function returns None	Missing return statement	Add return <value>

■ **Done When…**

Students can write a function that accepts a parameter and returns a value.

Students can call the function with different arguments and see different output.

Step 2 — Return Values & Multiple Parameters (0:25 – 0:40)

What to Say:

"A function can take multiple parameters — just separate them with commas. It can also return multiple values at once using a tuple. Let's build a mini calculator."

```
# ■■ Multiple parameters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def add(a, b):                      # two parameters
    return a + b

def divide(numerator, denominator):
    if denominator == 0:
        return None               # guard clause
    return numerator / denominator

# ■■ Returning multiple values ■■■■■■■■■■■■■■■■■■■■■■■■
def min_max(numbers):               # returns a tuple
    return min(numbers), max(numbers)

low, high = min_max([3, 1, 9, 4])  # unpack the tuple
print(low, high)                    # 1 9
```

■ **Instructor Tip**

Demonstrate that print() has no return value — print(print('hi')) shows 'None'.
This is a great moment to distinguish side-effects (print) from return values.

Step 3 — Default Arguments, *args, **kwargs (0:40 – 0:55)

What to Say:

"Sometimes you want a parameter to have a sensible value unless the caller overrides it — that's a default argument. When you don't know how many arguments you'll receive, Python gives you *args (a tuple) and **kwargs (a dictionary)."

```
# ■■ Default arguments ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def greet(name, greeting="Hello"):  # default value for greeting
    print(f"{greeting}, {name}!")

greet("Priya")                      # Hello, Priya!
greet("Arjun", "Namaste")           # Namaste, Arjun!

# ■■ *args — variable positional arguments ■■■■■■■■■■■■■
def total(*numbers):                # numbers is a tuple
    return sum(numbers)

print(total(1, 2, 3))               # 6
print(total(10, 20))                # 30

# ■■ **kwargs — variable keyword arguments ■■■■■■■■■■■■■
def profile(**info):                # info is a dict
    for key, value in info.items():
        print(f"{key}: {value}")

profile(name="Raj", city="Delhi", age=30)
# name: Raj   city: Delhi   age: 30
```

Common Errors — Step 3:

Error	Cause	Fix
SyntaxError: non-default after default	Required arg after default arg	Put defaults at end
*args received as list — wrong!	Students use list instead of *	Use *args syntax

Step 4 — Scope and the LEGB Rule (1:10 – 1:25)

What to Say:

"When Python looks up a variable name, it searches in a specific order: Local, then Enclosing, then Global, then Built-in. This is called the LEGB rule. Understanding it explains why some variables are accessible and others aren't."

LEGB Lookup Order:

Letter	Scope	Where	Example
L	Local	Inside the current function	x = 5 inside def foo()
E	Enclosing	Outer function (nested functions)	x in outer() visible in inner()
G	Global	Module-level (top of the file)	PI = 3.14 at top
B	Built-in	Python's built-in names	len, print, range, type

```
# ■■ LEGB in action ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
x = 'global'                        # Global scope

def outer():
    x = 'enclosing'                 # Enclosing scope
    def inner():
        x = 'local'                 # Local scope
        print(x)                    # local
    inner()
    print(x)                        # enclosing

outer()                             # local → enclosing
print(x)                            # global

# ■■ global keyword ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
counter = 0
def increment():
    global counter                  # modify global from inside function
    counter += 1
increment(); increment()
print(counter)                      # 2
```

■■ Warning

Using 'global' is usually a code smell. Teach students to prefer passing values as parameters and returning results instead of mutating globals.

Step 5 — Docstrings & Best Practices (1:40 – 1:55 Mini-Project)

What to Say:

"Good functions are short, do one thing, and document themselves with a docstring. Python reads the first string inside a function as its documentation — accessible via help()."

```
# ■■ Tip calculator mini-project ■■■■■■■■■■■■■■■■■■■■■■
def calculate_tip(bill_amount, tip_percent=18, num_people=1):
    """
    Calculate tip and total per person.

    Args:
        bill_amount  : Total bill before tip (float)
        tip_percent  : Tip percentage, default 18 (int)
        num_people   : People splitting the bill, default 1 (int)
    Returns:
        tuple: (tip_per_person, total_per_person)
    """
    tip = bill_amount * (tip_percent / 100)
    total = bill_amount + tip
    return round(tip / num_people, 2), round(total / num_people, 2)

tip, total = calculate_tip(1200, tip_percent=20, num_people=4)
print(f"Tip per person:   ■{tip}")   # ■60.0
print(f"Total per person: ■{total}") # ■360.0
```

■ SECTION 5 — SESSION WRAP-UP

Student Exit Checklist

■ I can define a function with def and call it

■ I understand parameters vs arguments

■ I can use return to send data out of a function

■ I can set default argument values

■ I can use *args to accept any number of positional arguments

■ I can use **kwargs to accept any number of keyword arguments

■ I can explain the LEGB rule and predict variable lookup

■ I know when (not) to use global

■ I can write a proper docstring for a function

Key Concepts Recap

Concept	One-line Summary
def	Defines a named, reusable block of code
Parameters / Args	Input data passed into a function
return	Outputs a value; without it, function returns None
Default args	Fallback values used when caller doesn't provide them
*args	Collects extra positional arguments into a tuple
**kwargs	Collects extra keyword arguments into a dict
LEGB rule	Local → Enclosing → Global → Built-in lookup order
global keyword	Allows modifying a global variable inside a function
Docstring	First string inside a function — read by help()

Next Day Preview — Day 6: Data Structures

Day 6 covers Python's built-in collection types: lists, tuples, dictionaries, and sets. Students will learn how to store, access, and transform groups of data — and write powerful one-liners with list/dict/set comprehensions.

Troubleshooting Notes

Symptom	Likely Cause	Quick Fix
UnboundLocalError inside function	global var used locally without global kw	Add 'global x' or pass as arg
Function always returns None	Missing return	Add explicit return statement
*args comes out as tuple, not list	Expected — it IS a tuple	Use list(*args) if list needed
Scope confusion in nested funcs	Missing nonlocal keyword	Use 'nonlocal x' in inner()

■ SECTION 6 — QUICK REFERENCE CARD

Function Syntax Patterns

```
# ■■ Minimal function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def say_hi(): print('Hi!')

# ■■ With parameters and return ■■■■■■■■■■■■■■■■■■■■■■
def add(a, b): return a + b

# ■■ Default argument ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def greet(name, msg="Hello"): print(f"{msg}, {name}!")

# ■■ *args  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def total(*nums): return sum(nums)

# ■■ **kwargs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def show(**kw):
    for k, v in kw.items(): print(k, v)

# ■■ Docstring ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def area(r):
    """Return area of a circle with radius r."""
    return 3.14159 * r ** 2
```

VS Code Shortcuts (Day 5)

Shortcut	Action
Ctrl+/ (Cmd+/)	Toggle comment on selected lines
F2	Rename symbol (function or variable name)
Ctrl+Shift+P → 'Outline'	View all function definitions in file
Hover over function call	See docstring tooltip in VS Code